

Heterogeneous-SSD 的 Device-Aware Index 分配

M08 組 蘇晟翔、李為新、周証恩

Code: <https://github.com/Vincent5201/NTHU-Advance-Storage-System-Project-m08>

一、研究介紹

1. 原動機與題目

原題目是「Mixed-Age SSD 的 Degradation-Aware Index 分配」。動機為資料中心中一個 storage pool 裡可能有不同壽命的 SSD 混一起，老 SSD 遇到 bloom filter 誤判的代價大於較新 SSD，但現行 KV store (如 RocksDB，使用 LSM Tree) 的 bloom filter 分配方法忽略了這個差異。因此想透過調整 bloom filter memory 分配策略來降低老 SSD 的 false positive rate (FPR)，以此來降低 tail latency。

2. 新動機與題目

我們發現原題目研究困難，為了有更全面的了解以及具體貢獻，將題目更新為「Heterogeneous-SSD 的 Device-Aware Index 分配」，進行兩個擴充：

- a. 原動機是 SSD 自然汰換形成的情境，我們另外考慮一種人為控制的情境，也就是基於成本原因，在最深容量最大的層使用較差/較舊/較便宜的 SSD。
- b. 對於新情境，我們加入「等級差異」的考量，分成高速 SSD 與普通 SSD。

最終我們想探討在一棵 LSM Tree 內的儲存裝置之間存在差異性的情況下，能否透過調整 bloom filter memory 的分配策略來改善整體效能。

3. 現有研究 (Monkey) 之缺陷

Monkey 透過嚴謹的數學證明提出「Bloom Filter 的分配應使誤判率(FPR)與層級深度成正比」，並實作在 LevelDB 上證實其效能。然而 Monkey 並未考慮到裝置差異的影響，特別是較差的裝置在深層(較高 FPR)時，更容易造成 I/O 瓶頸並急劇擴大 tail latency，因此本研究選擇建立於 Monkey 上提出 Device-Aware Monkey，將底層硬體的效能與狀態特徵納入優化模型中。

4. 具體情境設定

a. 自然形成的老化差異 (情境 A)

也就是原動機。一棵 LSM Tree 中，每個 SSTable 都有可能隨機放到老化的裝置上，並且此事與 SSTable 在第幾層無關。不過由於不太可能(也不應該)會有不同等異的 SSD 被隨意混在一起，因此本情境不考慮等級差異。

b. 刻意控制的裝置差異 (情境 B)

我們認為以下情境可以應用在實際情形中，因此將它加入實驗：一棵 LSM Tree 中，較淺的 1 到 N-1 層使用較好的 SSD，最深的第 N 層使用較差的 SSD。這樣設置是因為以下兩個原因：1. LSM Tree 中最深層的容量一般遠大於其他層的總和，實務上或許可以淺層使用較好 SSD，最深層則使用較差 SSD 來節省成本；2. 查詢要走到最深層必定要經過淺層，因此是淺層要用較好的 SSD。在此情境下我們考慮老化差異與等級差異。

5. 研究目標 (Research Questions)

- a. 仔細思考並分析上述题目的可行性
- b. 提出 Device-Aware Monkey 的分配方式
- c. 撰寫模擬程式碼來進行實驗，比較並分析不同分配策略的差異性
- d. 在 LevelDB / RocksDB 等真實資料庫上復現(未完成，見七、2 與七、3)

二、 思考與分析

1. 明確定義改善目標

Monkey 的目標是減少誤判的總 latency 不是減少 tail latency，因此在 Monkey 引入裝置係數也是減少總 latency，概念上是透過允許較多小 latency 來減少大 latency，而減少大 latency 對 tail latency 應該也更有幫助。那數學上是否能直接針對 tail latency 去做改善呢？經過實驗，我們尚未找到辦法，詳見七、1。

綜上所述，本研究提出的 Device-Aware Monkey 僅是「減少誤判造成的總 latency，觀察對於 tail latency 的影響」，並非是針對 tail latency 來改善。

2. 為何必須使用排隊系統來模擬

若要讓前述的觀察有意義，需要使用排隊系統來模擬，否則 tail latency 只是在

測量兩個裝置的讀取延遲機率分布，要改善 tail latency 就讓較差裝置的 FPR 降到非常低即可。而在排隊系統下，降低較差裝置的 FPR 會導致較好裝置的 FPR 升高產生較多查詢，查詢互相延遲 tail latency 反而更大，因此讓觀察有意義。

3. 非空查詢問題

我們發現 Monkey 只考慮 100%空查詢，因為他目標是減少誤判的總 latency。但我們要觀察 tail latency，僅考慮 100%空查詢的情境意義比較小，這種情境在現實中比較少見。然而引入實際查詢對 tail latency 會有特殊影響，原因如下：

- a. 假設是 50%空查詢，50%實際查詢，並且 FPR 是 2% (很高了)。
- b. 這樣會發出 51%查詢，相較於完美的 50% 差距僅有 1%，即便 FPR 從 2% 改善到 0.5%，這樣只會讓 p99 在機率分佈上平移一點點，除非系統已經進入很壅擠的情境(排隊擴大延遲)，否則 tail latency 不會有太大差別。
- c. 此外若資料就是在較差的裝置內，就一定要去讀取該裝置，tail latency 就會發生在該裝置上，無法避免。

綜上所述，我們認為「透過 Bloom Filter 的分配影響 tail latency」的做法僅在接近 100%空查詢的特殊情境才有效果，後續將透過實驗觀察它的效果遞減程度。

三、 Device-Aware Monkey 推導

1. Device-Aware Monkey 閉式解推導

我們設定 device 參數 d_i ，代表第 i 層的裝置等級，當 d_i 越大，代表裝置越差，並在 Monkey 公式中引入 d_i 重新推導一次。

首先總 memory 用量 M 定義如下，與原論文相同，但是將常數統一簡化為 C

$$M = \sum_{i=1}^L (-C \cdot \ln(P_i) \cdot T^i)$$

總懲罰函數 R 則插入了 device 參數 d_i ，目標是在滿足 M 的條件下最小化 R

$$R = \sum_{i=1}^L (d_i \cdot P_i)$$

為了求解，建立拉格朗日函數 (Lagrangian)

$$L = -C \sum_{i=1}^L (\ln(P_i) \cdot T^i) + \lambda \left[\sum_{i=1}^L (d_i P_i) - R \right]$$

對 P_i 求偏導數並令其為 0 以尋找極值點

$$\frac{\partial L}{\partial P_i} = -C \frac{T^i}{P_i} + \lambda d_i = 0$$

$$P_i = -\frac{C}{\lambda} \cdot \frac{T^i}{d_i}$$

由此可知， P_i 與 T^i 成正比，與 d_i 成反比，再將 P_i 關係式代約束條件回 M

$$M = \sum \left(-C \cdot T^i \cdot \ln \left(-\frac{C}{\lambda} \cdot \frac{T^i}{d_i} \right) \right)$$

利用對數性質 $\ln(ab) = \ln(a) + \ln(b)$ 展開並拆解整理，其中 $\sum T^i = N$

$$M = \sum \left(-CT^i \ln \left(-\frac{C}{\lambda} \right) \right) + \sum \left(-CT^i \ln \left(\frac{T^i}{d_i} \right) \right)$$

$$M = -CN \ln \left(-\frac{C}{\lambda} \right) + \sum \left(-CT^i \ln \left(\frac{T^i}{d_i} \right) \right)$$

$$\ln \left(-\frac{C}{\lambda} \right) = -\frac{M}{CN} - \sum \left(\frac{T^i}{N} \ln \left(\frac{T^i}{d_i} \right) \right)$$

兩邊取指數，將對數求和轉化為連乘（ $\prod$ ）形式（利用 $e^{a \ln x} = x^a$ 的性質）

$$-\frac{C}{\lambda} = e^{-\frac{M}{CN}} \cdot e^{\sum \frac{T^i}{N} \ln \left(\frac{d_i}{T^i} \right)} = e^{-\frac{M}{CN}} \cdot \prod \left(\frac{d_i}{T^i} \right)^{\frac{T^i}{N}}$$

最後，將求得的 $-\frac{C}{\lambda}$ 關係式代回原先的 $P_i = -\frac{C}{\lambda} \cdot \frac{T^i}{d_i}$ 中，即獲得最終的閉式解

$$P_i = e^{-\frac{M}{CN}} \cdot \prod \left(\frac{d_i}{T^i} \right)^{\frac{T^i}{N}} \cdot \frac{T^i}{d_i}$$

以上即為引入 Device-Aware 的 Monkey 公式，得到 FPR 除了與 T^i 成正比，還要與 d_i 成反比，也就是裝置越差， d_i 越大，FPR 就要越低，符合直覺邏輯。

2. 計算裝置係數 d_i

先前提到有以下兩種裝置差異，設定如下：

- a. 老化差異的 retry 機率 r_i ，每次讀取時有 r_i 的機率會讀取失敗要重讀。
- b. 等級差異的 low factor s_i ，次級 SSD 的基礎 latency 是高級 SSD 的 s_i 倍。

我們根據期望值的算法，將這兩者整合成以下公式：

$$d_i = (1 + \frac{r_i}{(1 - r_i)} * \text{penalty}) * s_i$$

考慮老化時將 s_i 設成 1，前半段就是讀取次數的期望值；考慮等級時將 penalty 設成 0，後段部分就是讀取速度的倍率；penalty 是額外設置的可控參數代表對老化的重視程度。透過此公式，我們可以僅考慮單一裝置差異，也可以合併考慮老化與等級兩種差異。最終 d_i 就是 Device-Aware Monkey 公式的裝置係數。

四、 實驗設定

本實驗透過一份模擬程式碼來模擬在 LSM-Tree 中讀取 SSD 的行為，其中包含 M/G/c 排隊理論模型與非同步 I/O 機制以求模擬結果盡量擬真。

1. LSM Tree 與情境設定

使用 4 層的 LSM Tree，設置放大率為 4，容量為 1、4、16、64，每次查詢一層只會讀取一個 SSTable，SSTable 會被放到兩種裝置上，具體設定如下：

- a. 自然形成的老化差異(A)：約有 30% (25/85) 的 SSTable 在老化裝置上，這些老化的 SSTable 的位置與層數無關，老化裝置的 read retry 機率較高。
- b. 刻意控制的裝置差異(B)：深層的 64 個 SSTable 在較差裝置上，老化差異(B-1)是 read retry 機率不同，等級差異(B-2)則是基礎讀取速度(low factor)不同。

2. 讀取延遲模擬

使用 M/G/c 排隊系統與非同步 I/O 來模擬，請求處理時間則結合 base latency、retry 機率 r_i 、等級倍率 s_i 來實際模擬讀取 LSM Tree 的行為，模擬方式如下：

- a. 系統依照 Poisson Arrival Process 產生查詢請求，每個查詢請求進入 LSM Tree 逐層搜尋，若 Bloom Filter 判定該層資料存在就向 SSD 發出實體 I/O。

- b. 所有實體 I/O 會進入同一個排隊系統等待可用通道，並且可能需要多次 read retry，其中讀取次數由 r_i 實際模擬，因此一次實體 I/O 的時間 t_{IO} 為：

$$t_{IO} = (\text{base latency} * s_i + \text{讀取次數} * \text{base latency} * s_i) + \text{排隊等待時間}$$

- c. 使用非同步 I/O，當一請求誤判多次可能會被其他請求干擾導致放大延遲。
(舉例：當請求 A 在 L2 發出實體 I/O a2 (誤判)，a2 執行時，請求 B 也進入並且在 L2 發出實體 I/O b2 (排隊在 a2 之後)。a2 執行完後 A 又在 L3 發出實體 I/O a3，a3 就得排在 b2 之後。導致 A 發生兩次實體 I/O，但總延遲時間受到了 B 的影響，從 $a2+a3$ 變成 $a2+b2+b3$ 。)
- d. 每個查詢延遲的完整計算方式如下。重點在於總延遲除了實際 I/O 時間，還會受到排隊系統的影響(如 b.跟 c.所述)導致延遲更大。

```
請求到達， $t_w = 0$  (延遲)
對於  $i = 1 \sim N$  層
  If 資料在第  $i$  層:
    進入排隊系統等待實體 I/O， $t_w += t_{IO}$ 
  Return  $t_w$ 
Else:
  If Bloom Filter 誤判:
    進入排隊系統等待實體 I/O， $t_w += t_{IO}$ 
Return  $t_w$ 
```

3. 分配策略

本研究比較三種分配策略。

- 平均分配(Mode 1)：所有 SSTable 有相同的 FPR
- Monkey 分配(Mode 2)：FPR 隨深度成正比
- Device-Aware Monkey 分配(Mode 3)：將裝置差異的因素引入 Monkey

4. 實驗方法

依照情境以及分配策略建立好 LSM Tree，進行 1,000,000 筆查詢請求來測試，實驗參數與評估指標如下：

- 關鍵指標：P99、P99.9、平均延遲，用以評估實驗結果的主要指標。
- 驗證指標：各層分配的 bits 數、FPR、false positive 數，以及總 throughput 與總 memory 用量，用以驗證實驗正確性。

- i. 程式碼會紀錄這些資料，但他們不是主要指標因此沒有放在報告裡。
- c. 主要變數：retry probability (老化差異)、low factor (等級差異)。
 - i. retry probability：未老化的 SSD 設置為 0.001，老化 SSD 則從 0.01 不斷提升到 0.4。
 - ii. low factor：高級 SSD 設置為 1，較差 SSD 則是從 1 提升到 6。
- d. 輔助變數：penalty、total memory、base latency、empty ratio、arrival rate、channel，用於模擬真實的讀取情境。
 - i. Penalty：對老化(tail latency)的重視程度。
 - ii. total memory = 850 代表平均分配下是 10 bits/key，total memory = 680 則代表平均分配下是 8 bits/key。
 - iii. empty ratio：代表空查詢的比例，主要實驗中皆設置為 1，另外會有部分實驗觀察不同 empty ratio 的影響。
 - iv. arrival rate：Poisson Process 的請求到達率，越高代表請求到的越快。
 - v. channel：主要是實驗在 channel=1 上，但由於現代 SSD 多具有高通道數，因此會透過部分實驗觀察高通道數下的變化。

此外本實驗是自行模擬，其絕對數值與現實不符，因此結果呈現會採用模式間的相對數值。例如「Mode 2 vs Mode 1」代表 Mode 2 相對於 Mode 1 的改進。

最終本實驗會在上述三種情境(A、B-1、B-2)下呈現以下實驗結果：

- a. 基本 Device-Aware Monkey 效果
- b. 不同 arrival rate 的差異
- c. 不同 channel 的差異
- d. 不同 penalty 的差異
- e. 不同總 memory 量的差異
- f. 非 100%空查詢

接下來我們會根據各項實驗結果進行分析、討論與總結，實驗數據眾多，最終結論請跳至 31 頁。

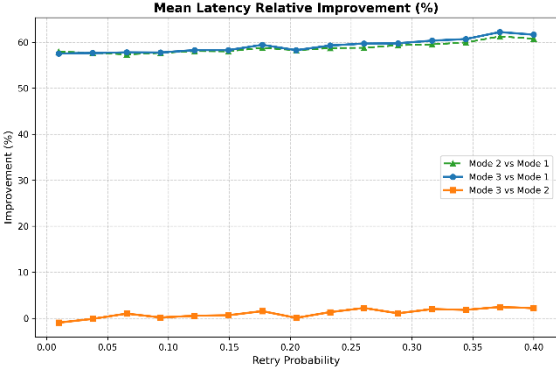
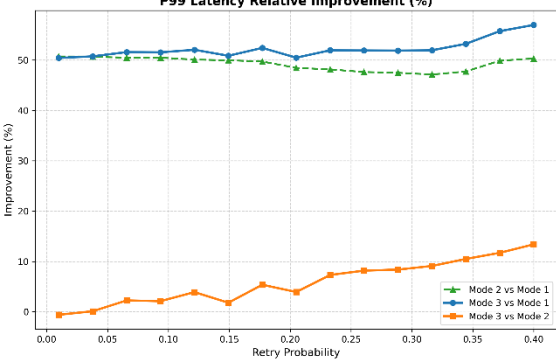
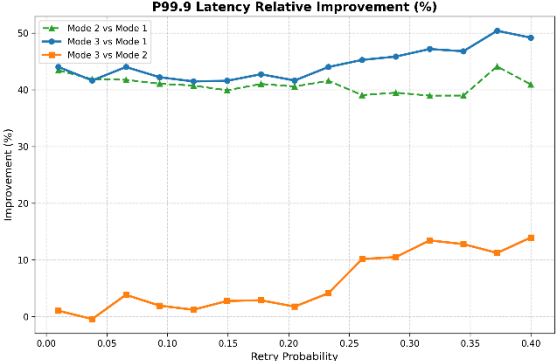
五、實驗結果與討論

1. 自然形成的老化差異 (情境 A)

設置 base latency=100、retry probability=0.01~0.4、empty ratio=1，觀察隨著 retry probability 漸大不同 Mode 的變化。

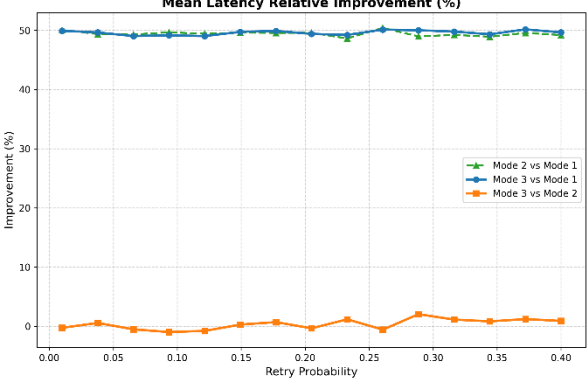
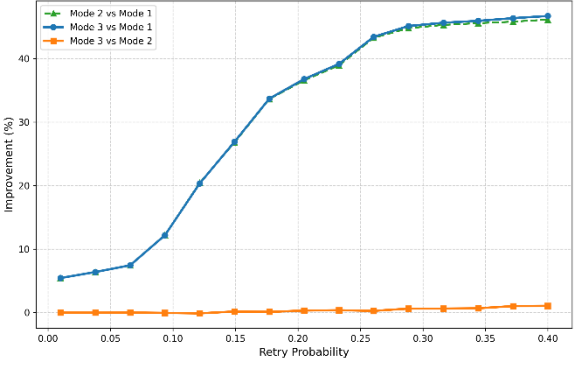
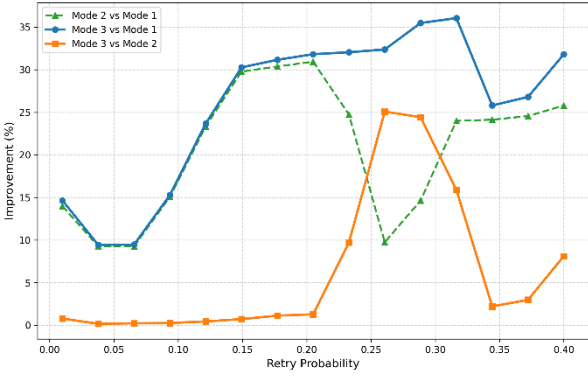
a. 基本 Device-Aware Monkey 效果

根據實驗 1-1，由於 Mode 2 在老化的深層配置較高 FPR，Mode 3 則避免了這點，因此隨著 retry probability 漸大 Mode 3 漸漸顯示出相較於 Mode 2 的優勢。並且 Mode 2/3 對於 Mode 1 都穩定有優勢。

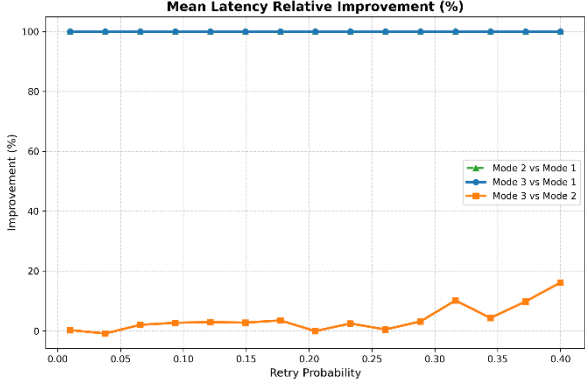
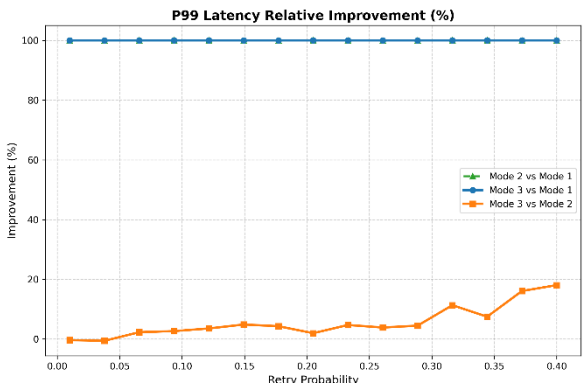
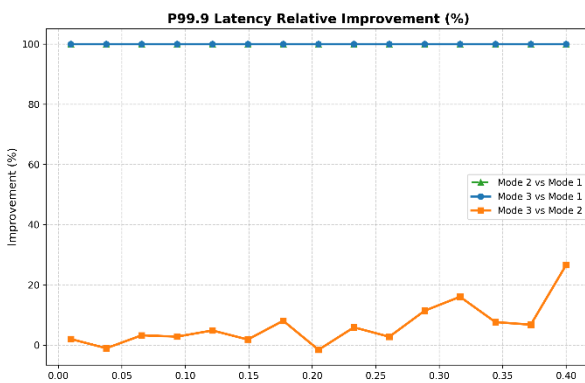
實驗 1-1	total memory	arrival rate	channel
參數設定	680	0.05	1
		平均 latency： Mode 2/3 相對於 Mode 1 都有明顯改善。Mode 3，相對 Mode 2 表現差距不大，並隨 retry probability 增加有微幅的改善。	
		P99： Mode 2/3 相對於 Mode 1 都有明顯改善。Mode 3 相對 Mode 2，在 retry probability 增加時更具優勢。	
		P99.9： 同 p99	

b. 不同 arrival rate 的差異

相較於實驗 1-1，1-2 降低了 arrival rate 使系統變得輕鬆，此時在平均 latency 與 p99 中 Mode 2/3 都差不多。而在 p99.9 中 Mode 3 突然有優勢推測是因為 tail latency 的階梯式特性，也就是在 retry probability 是 0.26 左右時，p99.9 在 Mode 2 進入了 retry 1 次的階段，Mode 3 則還在 retry 0 次，因此 Mode 3 出現明顯優勢，直到 Mode 3 也進入 retry 1 次後優勢被抹平，因為此時系統極為輕鬆，不需要排隊。

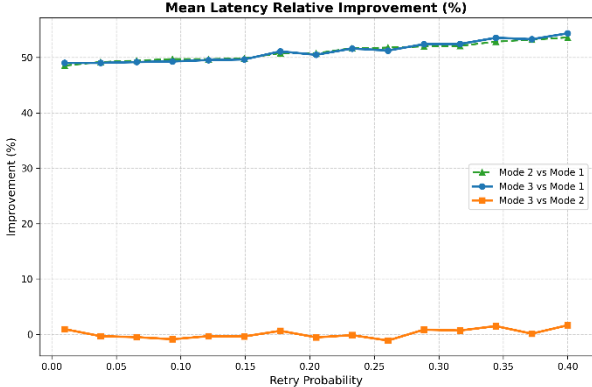
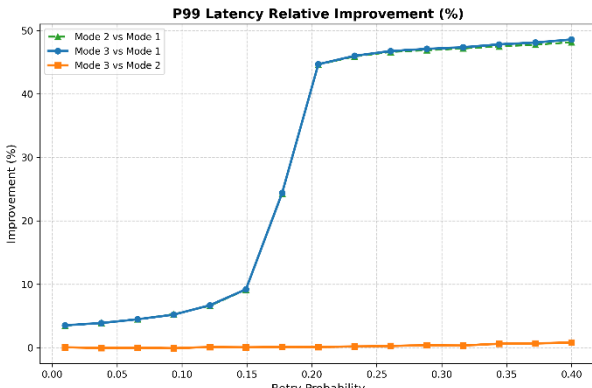
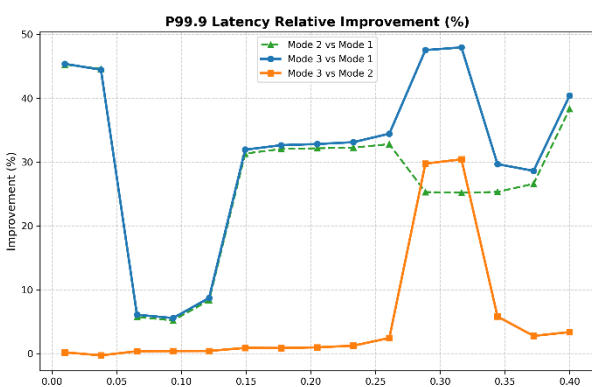
實驗 1-2	total memory	arrival rate	channel
參數設定	680	0.0095	1
		平均 latency： Mode 2/3 相對於 Mode 1 依然有明顯改善。	
		P99： Mode 2/3 相對於 Mode 隨 retry probability 改善逐漸上升。	
		P99.9： Mode 3 相對於 Mode 在 retry probability 小於 0.2 時相差不多，在 0.2 後則大幅上升再下降，並具有優勢。	

實驗 1-3 則是將 arrival rate 調高，此時 Mode 1 進入無限延遲的狀態。相較之下，Mode 2/3 成功存活。此外 Mode 3 對比 Mode 2 更能承受極限吞吐量的高壓環境，改善幅度提升到接近 20% (實驗 1-1 中僅有 10%左右)。

實驗 1-3	total memory	arrival rate	channel
參數設定	680	0.15	1
		平均 latency : Model1 進入了無限延遲的死結狀態。Mode 3，相對 Mode 2 隨 retry probability 增加有微幅的改善。	
		P99 : Mode 3，相對 Mode 2 隨 retry probability 增加有改善。	
		P99.9 : 同 P99	

c. 不同 channel 的差異

調高 channel 數，並且也相應的調高 arrival rate，由於高通道的特性，此時系統壓力極小，呈現跟 1-2 相似的趨勢。

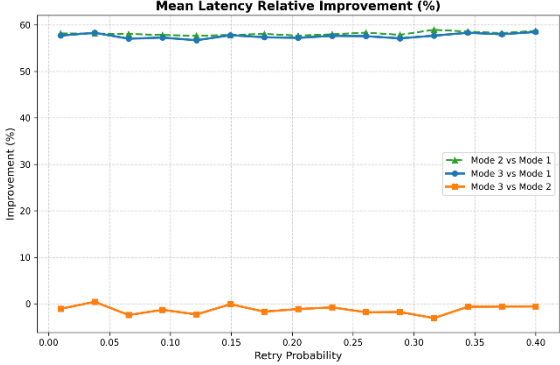
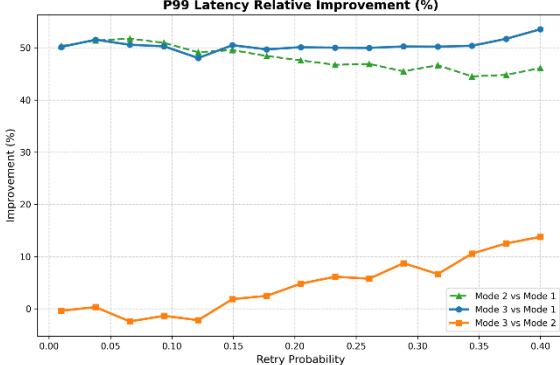
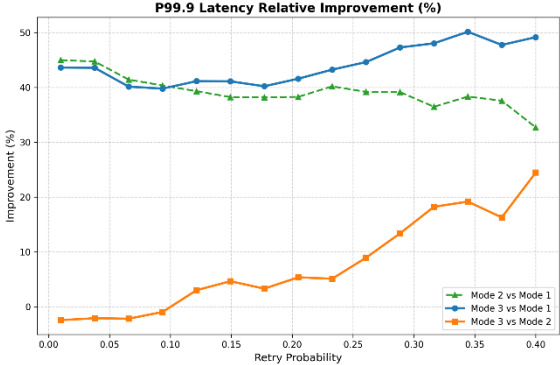
實驗 1-4	total memory	arrival rate	channel
參數設定	680	0.4	8
		平均 latency : Mode 2/3 相對於 Mode 1 都有明顯改善。Mode 3，相對 Mode 2 表現差距不大。	
		P99 : 同 Mean	
		P99.9 : Mode2/3 差距不大，主要在 retry probability 到達 0.25 後 Mode 3 較具優勢	

實驗 1-5 則是將 arrival rate 調高使系統進入壅擠狀態，Mode 3 相較於 Mode 2 就有明顯的改善，顯示在高 channel 下 Mode 3 仍能有所發揮。

實驗 1-5	total memory	arrival rate	channel
參數設定	680	1.6	8
Mean Latency Relative Improvement (%)		平均 latency : Mode 2/3 相對於 Mode 1 都有明顯改善。Mode 3 相對 Mode 2 在 retry 機率大於有較多改善。	
P99 Latency Relative Improvement (%)		P99 : 同 Mean	
P99.9 Latency Relative Improvement (%)		P99.9 : Mode2/3 差距不大，主要在 retry probability 到達 0.25 後 Mode 3 較具優勢	

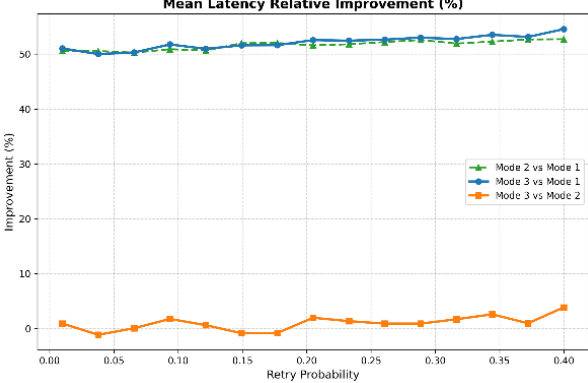
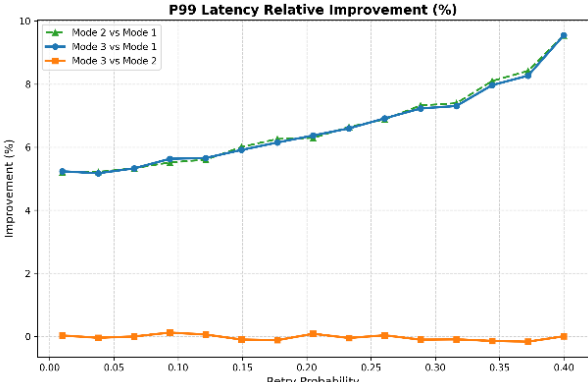
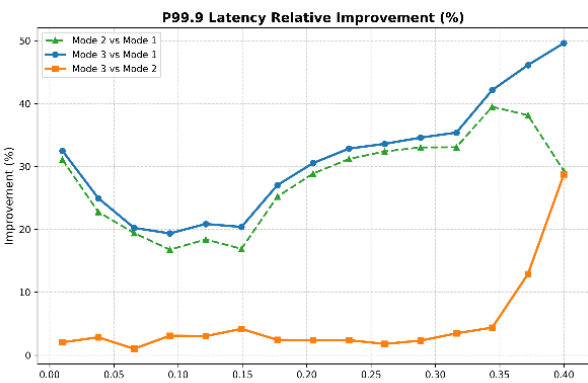
d. 不同 penalty 的差異

相較於實驗 1-1，1-6 將 penalty k 從 2 升到 4。由於更重視老化，使淺層的 FPR 提升，因此 Mode 3 平均 latency 比 Mode 2 略差，但換來的是在 p99 跟 p99.9，Mode 3 比 Mode 2 有更大幅的改善。

實驗 1-6	total memory	arrival rate	Penalty_k
參數設定	680	0.05	4
		平均 latency： Mode 2/3 都較 Mode 1 具有優勢。比較 Mode2/3，相較於實驗 1-1，系統持續處於「淺層 FPR 微幅偏高」的狀態。這使得進入實體 SSD 的總 I/O 數量常態性地多於 Mode 2，產生了輕微但持續的排隊代價。	
		P99： 隨 retry probability 變高時 Mode 3 有改善且幅度上升。	
		P99.9： 同 P99。	

e. 不同總 memory 量的差異

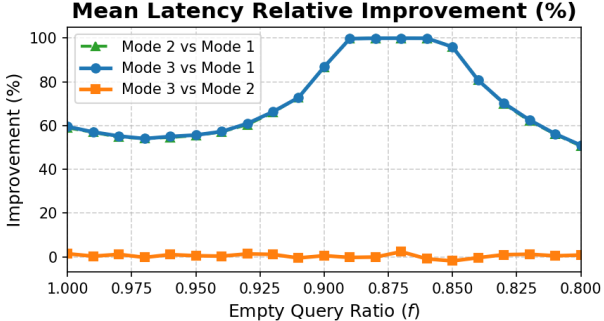
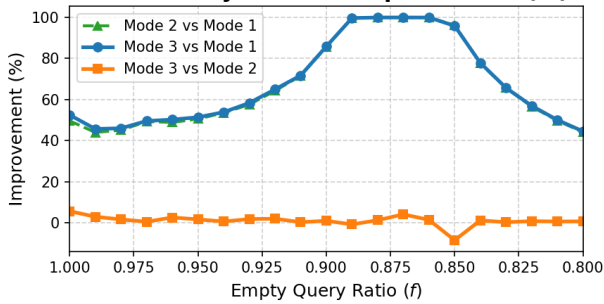
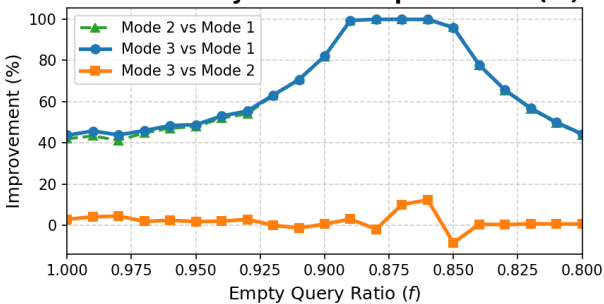
隨著 memory 從 680 上升到 850，FPR 會降低，系統變得輕鬆，因此 Mode 3 相對於 Mode 2 的優勢減少。至於 p99.9 中出現的波動推測與實驗 1-2 中提到的階梯式特性相同。

實驗 1-6	total memory	arrival rate	channel
參數設定	850	0.05	1
		平均 latency： Mode 2 與 Mode 3 差不多。	
		P99： Mode 2 與 Mode 3 差不多。	
		P99.9： Mode 3 在最老化時相較於 Mode 2 出現改善。	

f. 非 100%空查詢

本段將橫軸改成 empty ratio，設置 base latency=100、retry probability=0.2，用以實驗先前在二、3 提到的非空查詢情境。

在三項實驗結果中，Mode 2 與 Mode 3 都差不多，僅有 p99 一開始 Mode 3 比 Mode 2 略好，但隨著 empty ratio 降低，改善也逐漸降到 0。此外在 p99.9 的 0.875 附近出現一點波動，推測是因為 Mode 2 先進入了大量延遲的狀態因此表現下跌，Mode 3 則是較晚才進入大量延遲的狀態。

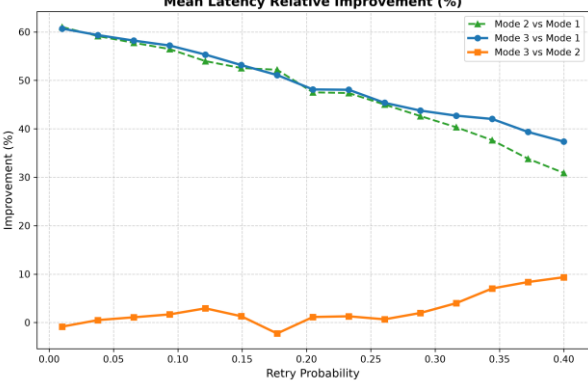
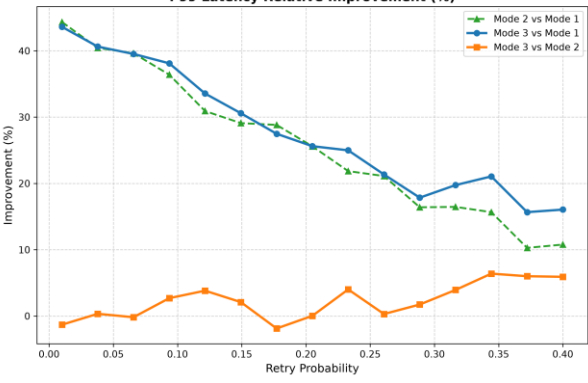
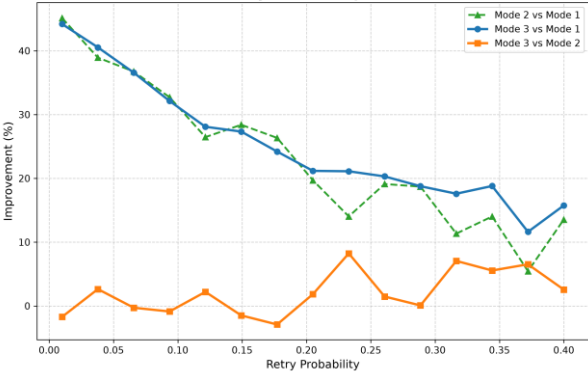
實驗 1-7	total memory	arrival rate	channel
參數設定	850	0.05	1
Mean Latency Relative Improvement (%) 		平均 latency： Mode 2 與 Mode 3 差不多。	
P99 Latency Relative Improvement (%) 		p99： Mode 3 一開始有略為比 Mode 2 好，之後兩者差不多。	
P99.9 Latency Relative Improvement (%) 		p99.9： Mode 3 在 0.875 的部分出現一點波動。	

2. 刻意控制的老化差異 (情境 B-1)

本段設置 base latency=100、retry probability=0.01~0.4、penalty k = 2。觀察隨著 retry probability 漸大不同 Mode 的變化。

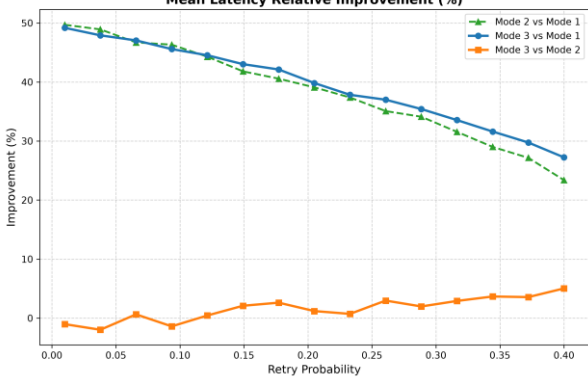
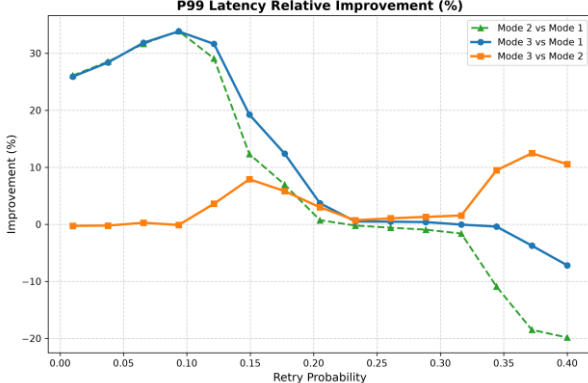
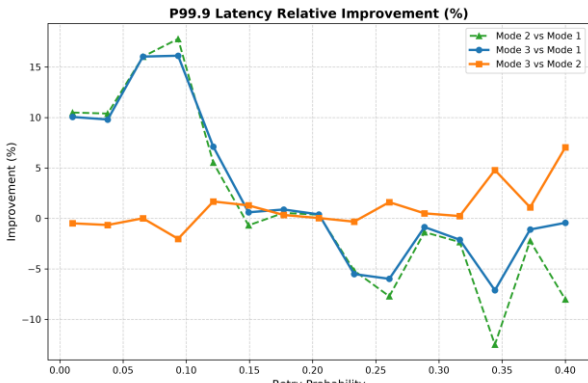
a. 基本 Device-Aware Monkey

實驗 2-1 顯示隨著老化增加，Mode 3 確實能改善 Mode 2，只不過改善幅度不大。此外 Mode 2/3 顯著優於 Mode 1。

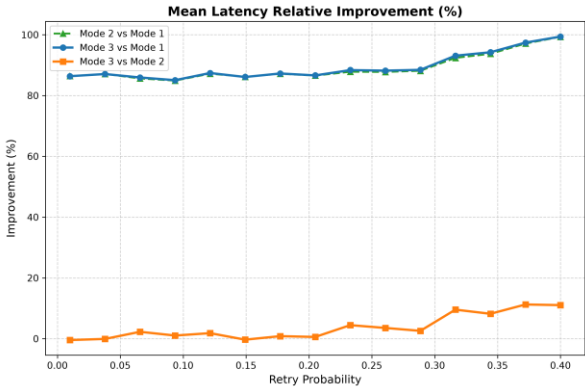
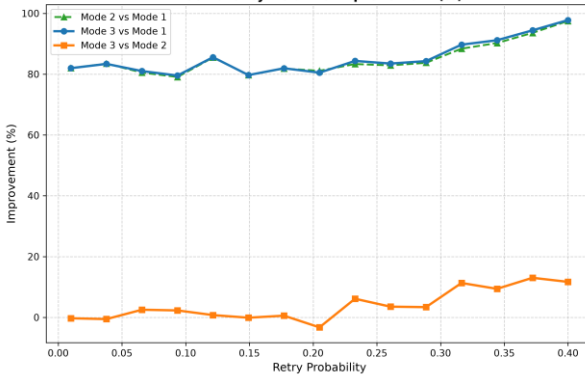
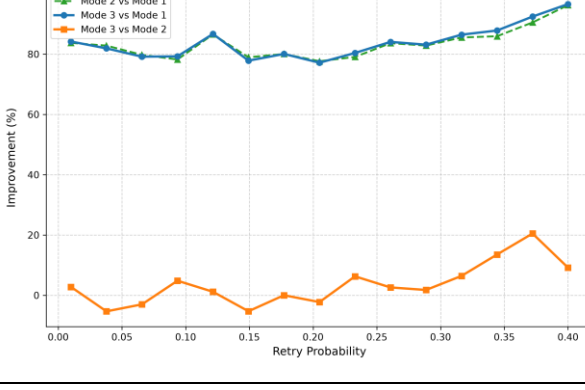
實驗 2-1	total memory	arrival rate	channel
參數設定	680	0.05	1
		平均 latency： Mode 2/3 相對於 Mode 1 都有明顯改善，隨著 retry 機率增加，改善幅度下降。Mode 3 在 retry 機率高時小贏 Mode 2。	
		P99： 同平均 latency。	
		P99.9： 同平均 latency。	

b. 不同 arrival rate 的差異

相比實驗 2-1，實驗 2-2 降低了 arrival rate，結果顯示 Mode 3 的改進變小了，甚至 Mode 2/3 在極度老化下不如 Mode 1，代表此時系統壓力極小，可以多分配一點 memory 給老化裝置。

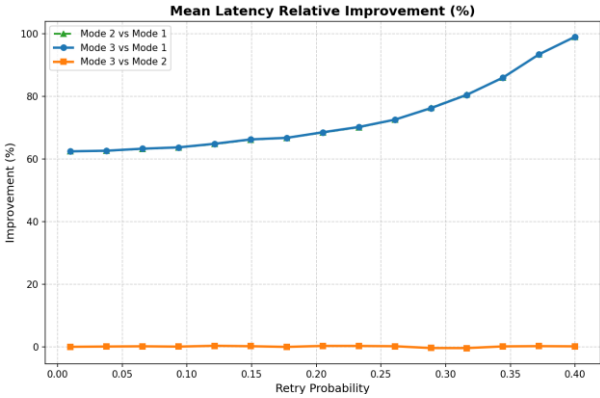
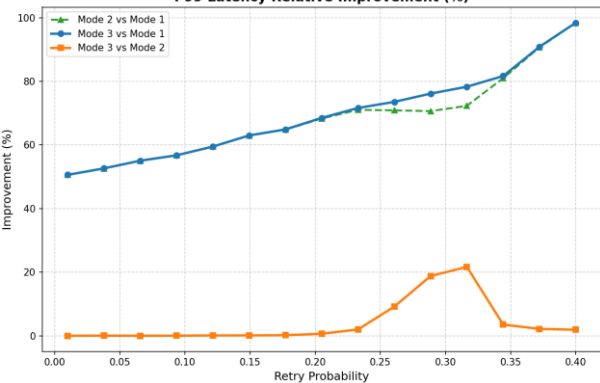
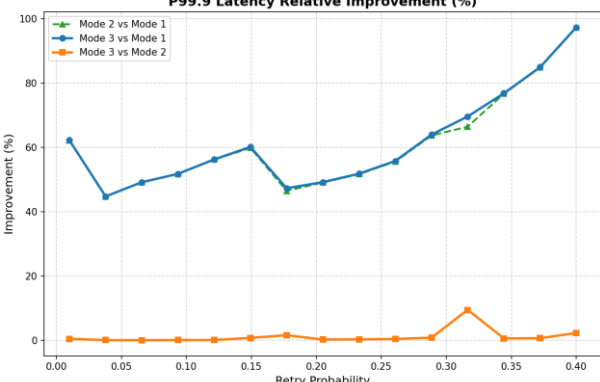
實驗 2-2	total memory	arrival rate	channel
參數設定	680	0.0095	1
Mean Latency Relative Improvement (%) 		平均 latency： 對 Mode 1 的改善變小，在相同 retry 機率下，improvement 大概降低 10%。	
P99 Latency Relative Improvement (%) 		P99： 在 retry 機率超過 0.2 時，反而不如 Mode 1，代表當系統壓力很小時，uniform 的配置在 retry 機率高的時候反而比較好，Mode 3 還是優於 Mode 2。	
P99.9 Latency Relative Improvement (%) 		P99.9： 同 P99。	

相比實驗 2-1，實驗 2-3 升高了 arrival rate，此時 Mode 3 相較於 Mode 2 的改善比 2-1 更多。

實驗 2-3	total memory	arrival rate	channel
參數設定	680	0.1	1
		平均 latency： Mode 2/3 相較於 Mode 1 都有大幅改善。	
		P99： 同平均 latency。	
		P99.9： 同平均 latency。	

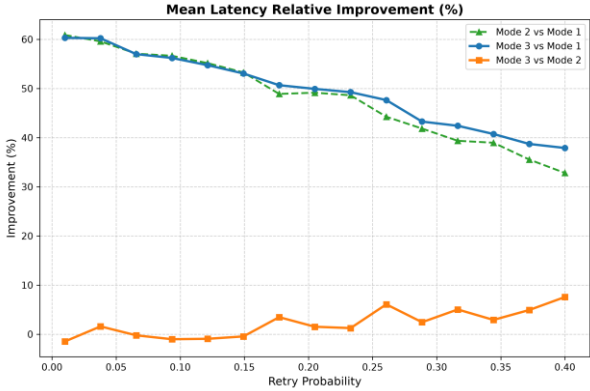
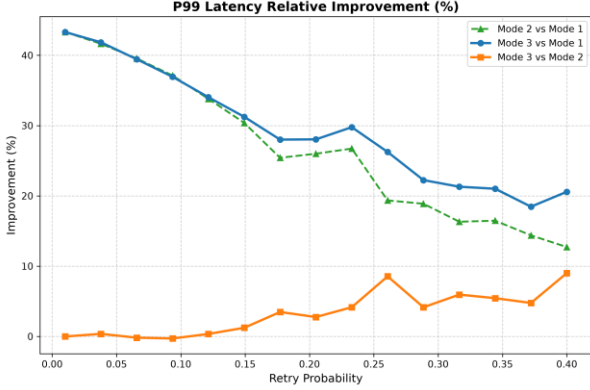
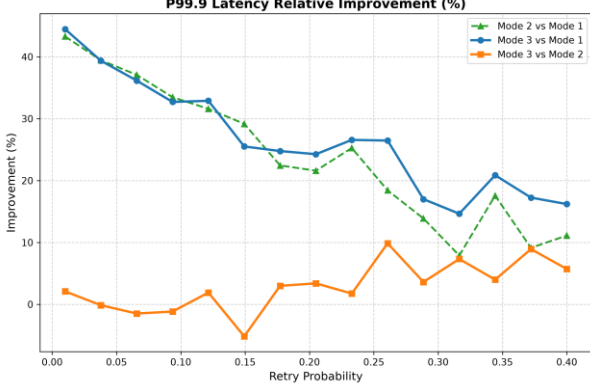
c. 不同 channel 的差異

相較於實驗 2-1，2-4 將 channel 數調高，arrival rate 也相應調高。由於高 channel 數系統不壅擠，因此 Mode 3 與 Mode 2 的效果差不多。有嘗試過更高的 arrival rate，但結果都與 2-4 差不多。

實驗 2-4	total memory	arrival rate	channel
參數設定	680	0.8	8
		平均 latency : Mode 2/3 相對 Mode 1 有改善，但 Mode 2 與 Mode 3 差不多。	
		P99 : 與平均 latency 相似。	
		P99.9 : 與平均 latency 相似。	

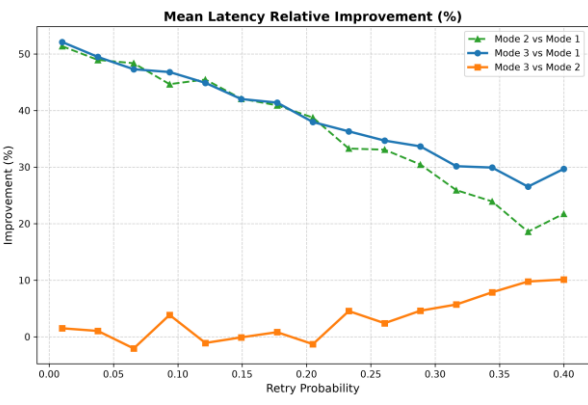
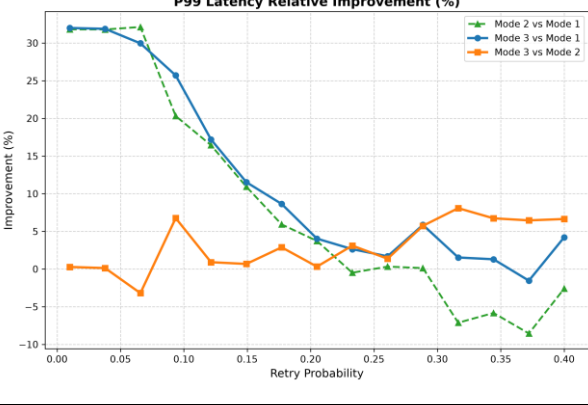
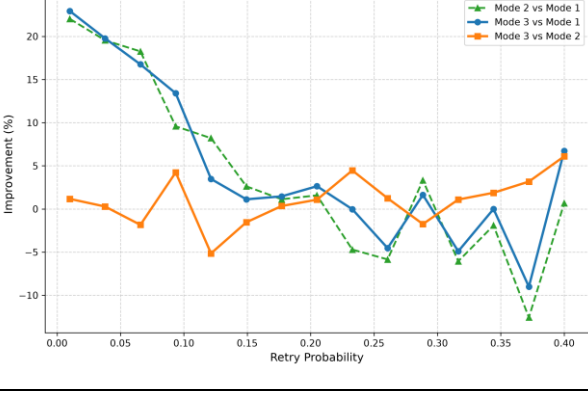
d. 不同 penalty 的差異

相較於實驗 2-1，2-5 把 penalty 從 2 調高到 4，在 mean latency 中 Mode 3 相對於 Mode 2 的改進有略微下降，但在 p99/p99.9 中改進有略微上升。

實驗 2-5	total memory	arrival rate	channel
參數設定	680	0.05	1
 Mean Latency Relative Improvement (%)		平均 latency : 分布跟實驗 2-1 差不多。	
 P99 Latency Relative Improvement (%)		P99 : 在 retry probability 變高時 Mode 2/3 差距被拉更開了。	
 P99.9 Latency Relative Improvement (%)		P99.9 : 同 P99。	

e. 不同總 memory 量的差異

相較於實驗 2-1，2-6 將 total memory 升到 850，整體趨勢與 2-1 差不多，但 FPR 降低後系統變的不壅擠，因此 Mode 2/3 相較於 Mode 1 的效果也降低。

實驗 2-6	total memory	arrival rate	channel
參數設定	850	0.05	1
		平均 latency： Mode 3 相較於 Mode 2 有改進但不多。	
		P99： Mode 2/3 相較於 Mode 1 的改進漸低，Mode 3 相較於 Mode 2 有改進但不多。	
		P99.9： 同 P99。	

f. 非 100%空查詢

本段將橫軸改成 empty ratio，設置 retry probability = 0.2、base latency = 100，用以實驗先前在二、3 提到的非空查詢情境。

實驗 2-7 在 p99 中，Mode 3 的改善會隨著 empty ratio 降低而減少，這點與先前的推測一致，p99.9 則因為實際查詢的引入，Mode 2/3 差不多。

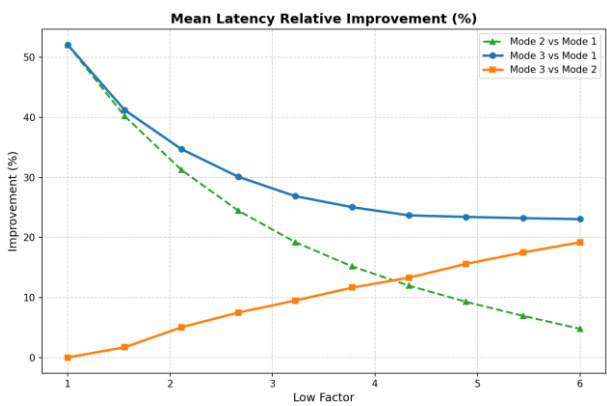
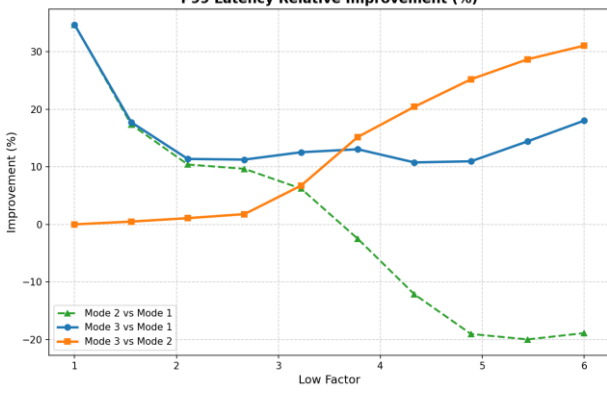
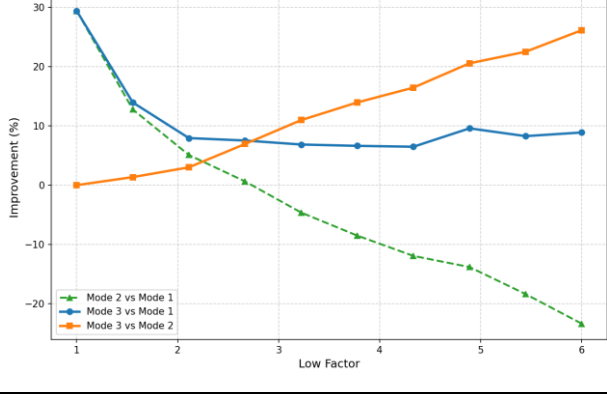
實驗 2-7	total memory	arrival rate	channel
參數設定	680	0.02	1
Mean Latency Relative Improvement (%) 		平均 latency： Mode 2 與 Mode 3 差不多	
P99 Latency Relative Improvement (%) 		P99： Mode 2/3 相較於 Mode 1 的改進漸低，Mode 3 相較於 Mode 2 有改進但漸漸降低。	
P99.9 Latency Relative Improvement (%) 		P99.9： Mode 2 與 Mode 3 差不多	

3. 刻意控制的等級差異 (情境 B-2)

設置 base latency=50、low factor=1~6、empty ratio=1。也就是 latency 會從 50~300，觀察隨著等級差異漸大不同分配策略的變化。

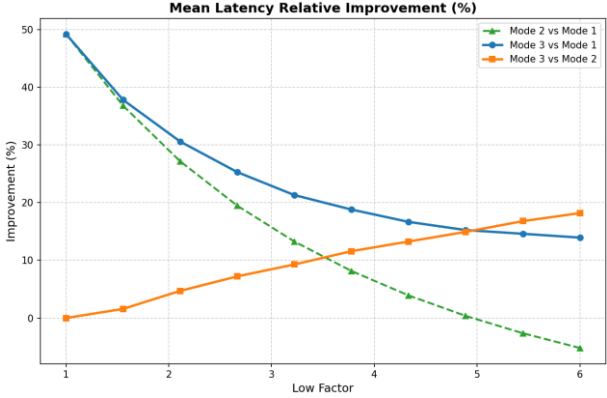
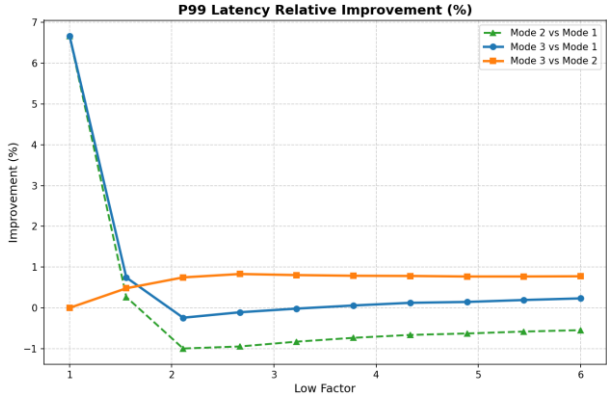
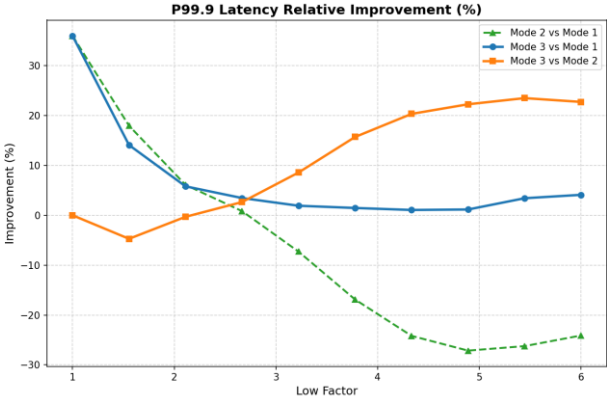
a. 基本 Device-Aware Monkey 效果

根據實驗 3-1，隨著 low factor 漸大，由於 Monkey 在深層配置較高的 FPR，導致 Mode 2 的 p99 與 p99.9 效果變差，特別是變成比 Mode 1 還差。Mode 3 則沒有上述問題，平穩保持著一定幅度的改善。

實驗 3-1	total memory	arrival rate	channel
參數設定	680	0.05	1
			
平均 latency： Mode 2/3 相對於 Mode 1 都有明顯改善。Mode 3 隨 low factor 越大，相對 Mode 2 改善越大。			
			
P99： Mode 2 在 low factor 大時效果急遽下降，變成比 Mode 1 還差。Mode 3 則沒有以上問題，維持穩定表現。			
			
P99.9： 同 p99			

b. 不同 arrival rate 的差異

相較於實驗 3-1，3-2 降低了 arrival rate 使系統變得輕鬆。結果顯示三種 Mode 的間的差距變小了。代表當系統壓力很小時，會退化回單純量測 SSD 讀取的機率分布。

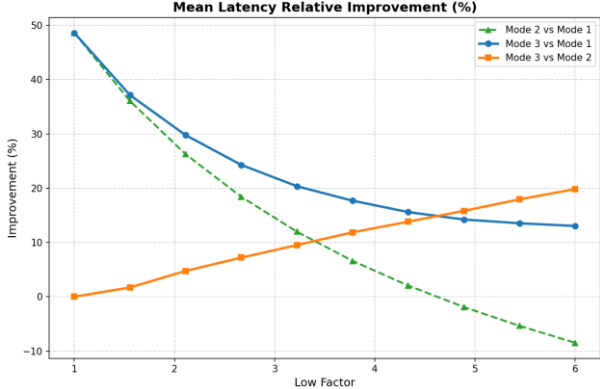
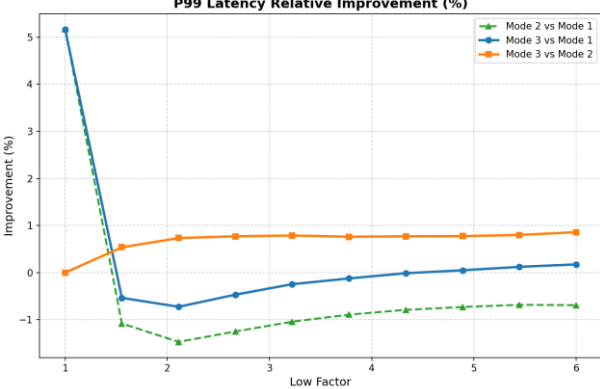
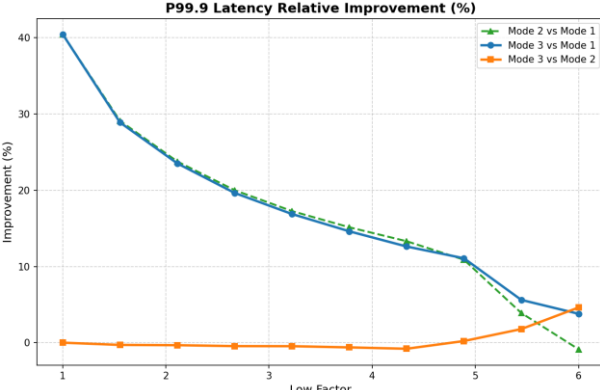
實驗 3-2	total memory	arrival rate	channel
參數設定	680	0.0095	1
		平均 latency： 對 Mode 1 的改善變小。	
		P99： 都在 0% 附近基本差不多。	
		P99.9： 與實驗 3-1 類似，但效果變小	

相較於實驗 3-1，3-3 升高 arrival rate 使系統變擁擠。當 low factor 漸大，排隊系統擠爆了，latency 變成是數百甚至數千倍，進入了無限延遲。根據結果，Mode 1 先進入無限延遲，接著 Mode 2 也在 low factor=3.7 左右進入無限延遲，Mode 3 則在 low factor=4.9 左右才進入無限延遲，並且表現也穩定比 Mode 2 好。結果顯示在排隊系統極為壅擠時，Mode 3 較慢進入無限延遲狀態，暴增幅度也比其他 Mode 少。

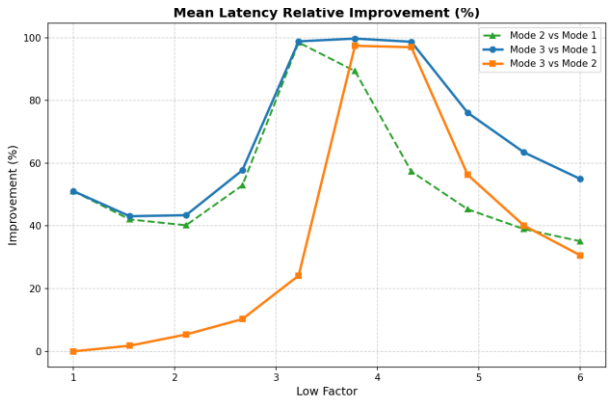
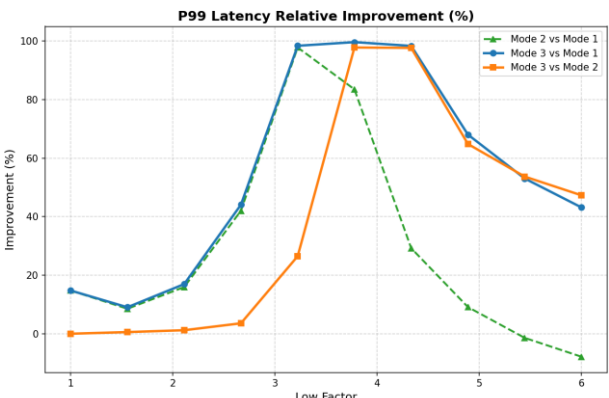
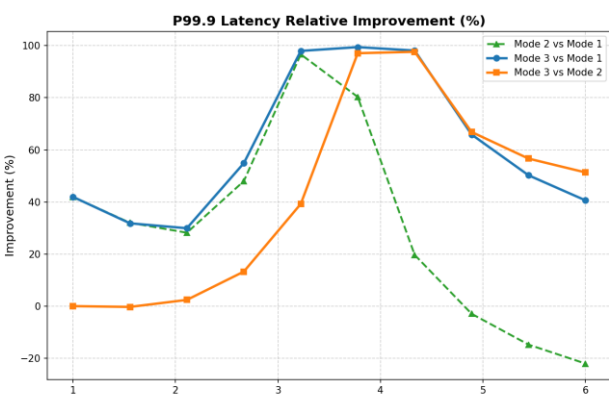
實驗 3-3	total memory	arrival rate	channel
參數設定	680	0.15	1
Mean Latency Relative Improvement (%)		平均 latency : Mode 3 相較於 Mode 1 都有顯著改善。Mode 3 相較於 Mode 2 先大幅上升再下降。	
P99 Latency Relative Improvement (%)		P99 : 同平均 latency。	
P99.9 Latency Relative Improvement (%)		P99.9 : 同平均 latency。	

c. 不同 channel 的差異

相較於實驗 3-1，3-4 將 channel 數調成 8 倍，arrival rate 也調成 8 倍。從平均 latency 跟 p99 可以看出由於高 channel 數，3-4 相較於 3-1 更不壅擠，因此 Mode 3 的效果降低。

實驗 3-4	total memory	arrival rate	channel
參數設定	680	0.4	8
		平均 latency： Mode 2/3 相較於 Mode 1 都有顯著改善。Mode 3 相較於 Mode 2 改善漸大。	
		P99： 三者差距不大	
		P99.9： Mode 2/3 相較於 Mode 1 的改進漸小。	

相較於實驗 3-4，3-5 將 arrival rate 再度調高進入壅擠的狀態，實驗結果與 3-3 類似。顯示即便是高 channel 數，只要進入擁擠狀態，Mode 3 也較慢進入無限延遲狀態，暴增幅度也比其他 Mode 少。

實驗 3-5	total memory	arrival rate	channel
參數設定	680	1.2	8
		平均 latency : Mode 2/3 相較於 Mode 1 都有顯著改善。Mode 3 相較於 Mode 2 改善漸大。	
		P99 : 同平均 latency。	
		P99.9 : 同平均 latency。	

d. 不同總 memory 量的差異

相較於實驗 3-1，3-6 將 total memory 升到 850。整體趨勢與 3-1 差不多，但在 p99 中 Mode 3 相比於 Mode 2 有極大幅的 60%改進(3-1 只有 30%)。這是因為在 3-1 中 FPR 較高，因此三個 Mode 都已經進入有些壅擠的狀態，然而 3-6 時 FPR 降低，剛好 Mode 2 已經略為壅擠，但 Mode 3 並不壅擠，因此在 p99 產生了大幅改進。

實驗 3-6	total memory	arrival rate	channel
參數設定	850	0.05	1
Mean Latency Relative Improvement (%)		平均 latency : Mode 2/3 相較於 Mode 1 都有顯著改善。Mode 3 相較於 Mode 2 改善漸大。	
P99 Latency Relative Improvement (%)		P99 : Mode 2 在 low factor 大時效果急遽下降，變成比 Mode 1 還差。Mode 3 則沒有以上問題，維持穩定表現。	
P99.9 Latency Relative Improvement (%)		P99.9 : Mode 2 在 low factor 大時效果急遽下降，變成比 Mode 1 還差。Mode 3 則沒有以上問題，維持穩定表現。	

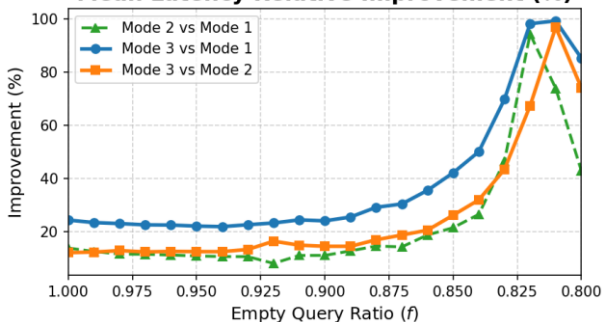
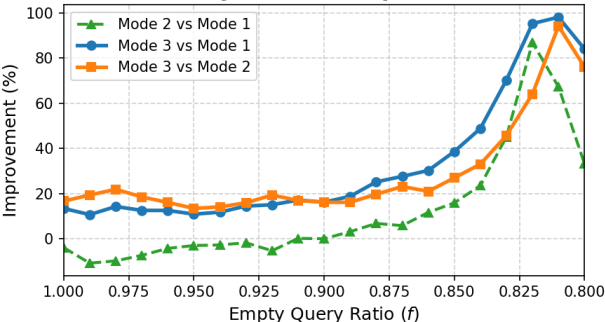
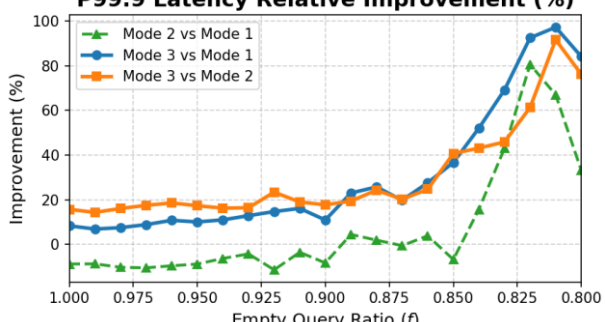
e. 非 100%空查詢

本段將橫軸改成 empty ratio，設置 base latency=50、low factor=4，用以實驗先前在二、3 提到的非空查詢情境。

實驗 3-7 呈現的是一般狀況下，Mode 3 帶來的改善會隨著 empty ratio 降低而減少，這點與先前的推測一致。

實驗 3-7	total memory	arrival rate	channel
參數設定	850	0.05	1
Mean Latency Relative Improvement (%)		平均 latency： 一開始有較大改進，之後改進幅度漸小。	
P99 Latency Relative Improvement (%)		P99： 僅一開始有較大改進。	
P99.9 Latency Relative Improvement (%)		P99.9： 一開始有較大改進，之後改進幅度漸小。	

相較於實驗 3-7，3-8 則是將 memory 用量變低，FPR 變高，因此比較壅擠，在 empty ratio 變低後，系統就會因為讀取 I/O 的請求太多而進入無限延遲狀態。這部分與先前的 3-3 類似，Mode 3 較慢進入無限延遲狀態，暴增幅度也比其他 Mode 少。

實驗 3-8	total memory	arrival rate	channel
參數設定	680	0.05	1
Mean Latency Relative Improvement (%) 		平均 latency： Mode 2/3 的改善在末尾暴增。	
P99 Latency Relative Improvement (%) 		P99： 同平均 latency。	
P99.9 Latency Relative Improvement (%) 		P99.9： 同平均 latency。	

4. 實驗結果結論

- a. 大部分情況下，Mode 2/3 都顯著優於 Mode 1。
- b. 在排隊系統較壅擠時，比較能凸顯 Mode 3 相較於 Mode 2 的優勢。
- c. 裝置的等級差異較能凸顯出 Mode 3 的有效性，在老化差異的實驗結果中，Mode 3 與 Mode 2 的實驗結果差距比較小。
- d. 排隊系統壅擠時，Mode 2 可能會先因為排隊壅擠而進入極高延遲的狀態，此時 Mode 3 相對於 Mode 2 就會有比較大的改善。
(memory、channel、arrival rate 都是影響壅擠與否的因素)
- e. 即便是在現今高 channel 數的情況下，只要進入較壅擠的狀態，Mode 3 相較於 Mode 2 仍能有所改進。
- f. 對於公式中的 penalty 參數，把它適當調高可以更好的改善 tail latency，但是會損害到 mean latency。
- g. 在 empty ratio 降低後，原則上 Mode 3 的改善會漸小，與推測一致。

六、 結論

本研究針對 LSM-Tree 下的異質 SSD 儲存環境，重點探討了改善 tail latency 的可行性，而後提出並驗證了 Device-Aware Monkey 的 Bloom Filter 分配策略。

首先，我們仔細分析後明確指出了「透過 Bloom Filter 分配來改善 tail latency」這項動機的兩大局限：1. 目前僅是在改善總 latency 並觀察 tail latency 的變化，不是針對 tail latency 直接做改善；2. 在 empty ratio 較低時此動機的意義會降低。但我們仍指出在透過在排隊系統下實驗可以獲得較有意義的實驗與觀察。

接著，我們在理論上擴充了原有的 Monkey 公式，提出 Device-Aware Monkey，證明在追求總誤判延遲最小化的前提下，Bloom Filter 的誤判率除了與層級大小成正比，還要與裝置係數成反比。代表性能較差的裝置應被分配更多的記憶體以降低其誤判率，並且我們為老化與等級差異訂定了裝置係數的計算方式。

最後，我們設計了盡可能擬真的模擬實驗，其中正向的實驗結果顯示：

1. 在排隊系統較壅擠下，Device-Aware Monkey 能改善 Monkey。
2. Device-Aware Monkey 與 Monkey 大部分都顯著優於平均分配策略，而在高壅擠情形下，Monkey 可能會因為深層的高 FPR 變得比平均分配還差，但特別的是 Device-Aware Monkey 不會，可以維持穩定的表並承受較大壓力，。

然而實驗結果也顯示 Device-Aware Monkey 的侷限性：

1. 在老化差異的實驗中(情境 A、B-1)，Device-Aware Monkey 與 Monkey 的表現差距比較小，僅在裝置差異(情境 B-2)下有明顯的效果差距。
2. 處於非壅擠狀態時，Device-Aware Monkey 與 Monkey 的差異極為有限。
3. 當 empty ratio 降低，Device-Aware Monkey 與 Monkey 的差異也隨之降低。

總結而言，Device-Aware Monkey 並非適用於全場景的泛用型方案，然而在面臨高壓壅擠的特定情境下，Device-Aware Monkey 能有效避免 Monkey 策略中深層高 FPR 引起的排隊延遲發散，展現出較佳的系統抗壓性與穩定度。

七、 其他

本段紀錄嘗試過哪些事情但是沒有效果或是沒有成功，以及分析為什麼。

1. 無法透過 Bloom Filter 分配針對性的改善 tail latency

先前已提到 Monkey 並非是在改善 tail latency，因此我們嘗試過數種方法來針對性的改善 tail latency (在 100% 空查詢下)，包含：

- a. 讓較差 SSD 的 FPR 降到 1% 以下(假設目標是 p99)，剩下用 Monkey 分配。
- b. 將該裝置的 tail latency(p99) 直接作為裝置係數。
- c. 基於 tail latency 跟變異數(variance)的特性，將裝置係數平方。

推導後發現，在數學上最終的答案都會回到 a。我們也有對這三種方法進行實驗，但結果都比 Device-Aware Monkey 差，因為分配過多 memory 給深層了。

2. 無法在 RocksDB 有效引入 Monkey 機制

具體做法是設置一個資料數使其大約填滿到 L4 中有 256 個 SSTable，就可以在程式碼中直接指定各層的 bit-per-key 資訊來簡單模擬 Monkey (無法 100% 準確但大致準確)，有透過內建 log 以及自行新增 log 來確認運行狀況符合預期。

我們在 100% 空查詢下測試，卻發現 Monkey 的表現甚至比原 Uniform 分配更差，特別是 false positive 數反而變得更多。

由於 RocksDB 整體機制複雜，我們無法準確確認具體原因。若不考慮實作錯

誤，目前唯一的猜測是由於 tired compaction，RocksDB 會在深層會需要讀取數個 SSTable 而非僅需要一個，導致深層的高 FPR 被放大。

之後考慮到 RocksDB 整體機制過於複雜，並且沒有明確的動機與目標，就沒有再嘗試新增其他機制進 RocksDB。

3. 無法在 LevelDB 有效引入 Monkey 機制

在原 Monkey 論文中，作者在 LevelDB 上實作了 Monkey 機制並證實他的有效性，並且驗證了即使有使用 Block Cache，Monkey 相對於 Uniform 分配仍有改善，因此我們想改用 LevelDB 進行嘗試。

然而我們將 Monkey 植入 LevelDB 後，但其效果跟 Uniform 一樣，我們尚未能確認其原因。